

Saday Wellness — Solo Agentic Build Estimate & Internal Audit

Document: docs/Saday_Solo_Agentic_Build_Estimate.html · **Date:** 2026-05-05 · **Prepared for:** Akash (CTO & practicing psychiatrist) · **Companion to:** data-model.md v1.1, intake-spec.md v1.0, stack-decision.md v1.0, Saday_Action_Plan_by_Claude.md · **Purpose:** a single document that quantifies the cost, leverage, risk, and uncompressible blockers of a solo + agentic-codegen build path on the locked stack (Next.js 14 + Supabase Postgres + RLS-first + TypeScript), so it can be compared head-to-head against the vendor quote (₹2L / 60 days / Vue + Laravel + MySQL) and an alternative hybrid path.

This is an internal estimating document. It is biased toward honesty, not toward selling a path. Wherever an estimate is optimistic, it is flagged. Wherever a number depends on a calendar-blocking dependency outside Akash's control (CF time, MSG91 template approval, pen-test scheduling), it is called out separately and *not* reduced by codegen leverage.

1 • Executive summary

Path	Wall-clock	Cash out	What you own at the end	Risk profile
Vendor as quoted ₹2L / 60 working days / Vue + Laravel + MySQL	~12 calendar weeks	₹2L + ~₹3k/mo infra (under-spec)	Demo-shaped MVP on a stack you didn't pick. 3 locked spec decisions silently violated. B6 (SOAP body deny) is at app layer not DB layer. Audit log = INSERT-only table, not Merkle-anchored.	High. Compliance posture regression vs spec. DPDP / MHA exposure if scrutinised.
Vendor with renegotiated stack + scope Postgres-first, paise, C-SSRS, B-series intact	~16–20 calendar weeks	~₹3–5L renegotiated + ₹15–25k/mo infra	Spec-true MVP on Postgres. You don't write a line; you spec-review every PR.	Medium. Vendor competence still unverified on the harder bits (RLS, audit cryptography, Vault).
Solo + agentic codegen You, Claude Code/Cursor, 25h/week	24–28 calendar weeks	~₹15–25k/mo infra + pen-test ₹40–80k	Spec-true MVP. Full stack ownership. Bus factor = 1 (you).	Medium. Two-hat clinical bandwidth risk + review fatigue + RLS debugging tax.
Hybrid (recommended) You build safety-critical; contractor builds dashboards	~14–16 calendar weeks	~₹2–3L contractor + ₹15–25k/mo infra + pen-test ₹40–80k	Spec-true MVP. You own all safety-critical code. Contractor owns well-trodden surfaces.	Low–medium. Best balance of correctness, speed, and bus factor.

Headline number: Solo + agentic on the locked stack lands at **~110–140 dev-days, 24–28 calendar weeks at 25 hours/week**. The single biggest variable is not your typing speed; it is **CF availability** (clinical sign-off, Hindi review) and **external dependencies** (MSG91 template approval, Razorpay live activation,

pen-test scheduling). These are uncompressible by codegen and consume 4–6 of those calendar weeks regardless of how fast you code.

2 · Why this stack is materially better for agentic codegen

Stack choice is not aesthetics. The locked stack (Next.js 14 + Supabase Postgres + RLS-first + TypeScript end-to-end) was picked deliberately in `stack-decision.md` ; agentic codegen amplifies *every one of those choices*. The vendor's pivot to Vue + Laravel + MySQL undoes the same amplifications.

Stack property	Why codegen produces fewer landmines
TypeScript end-to-end	Hallucinated column names and shape mismatches fail at <i>compile time</i> , not at runtime in a clinic. Single biggest quality lever an agentic dev can hold.
supabase gen types typescript	Schema is the source of truth. Types regenerate after every migration. Codegen consumes the generated types and stops inventing fields it has not seen.
RLS policies are pure SQL	A policy is 5–15 lines, easy for an LLM to draft, easy for a human to read, fails closed by default. Compare to Laravel policies — 200+ controller files where forgetting <code>\$this->authorize()</code> once equals a leak.
Server Components + Server Actions	Eliminates a layer (REST API route + client fetch + duplicated validation). Less surface for codegen to misalign client and server.
Single language across surfaces	Frontend, server actions, edge functions all in TS. Codegen's context window stays coherent across files; PHP+JS forces two mental models.
Managed Supabase + Vercel/Cloudflare	Codegen does not write nginx, supervisor, queue worker daemons, redis tuning, log rotation, php-fpm pool sizing. That work disappears entirely. Saves ~1 hour/week of ops indefinitely.
Drizzle / Supabase-js typed query builders	Column references are statically checked. The single most common LLM SQL error — <code>WHERE user_id = ...</code> when the column is <code>userId</code> — fails compile.
Supabase Vault + app_secret() helper	Crypto key custody is a one-time setup; codegen never sees the key. On Laravel + APP_KEY, codegen will routinely write <code>pgp_sym_encrypt('PHI', env('KEY'))</code> patterns and tempt fate.
RLS attack tests	The <code>data-model.md §7.4</code> adversarial test suite runs against the database, not against the application. Codegen can write 100 controllers; if RLS holds, it does not matter that 3 of them forgot to authz. This is the single biggest reason the spec was written Postgres-first.

The most under-rated point. RLS converts "did I remember to authz this controller?" from a per-route human review (which fatigues a solo dev after week 4) into a single SQL test suite. That is the architecture paying you back exactly when codegen's failure mode bites — 80% review acceptance + 20% rubber-stamping after fatigue sets in.

3 · Where codegen struggles on this stack (honest)

Symmetry: it is not a free lunch. Six places where agentic codegen on this exact stack costs more than naive estimates assume:

1. **RLS debugging tax.** "Why is this query returning empty when I'm clearly logged in?" Famous time-eater. When a test fails, codegen's reflex is to *broaden* the policy; you must enforce the discipline of "narrow first, prove the case, then accept." Budget 1.2–1.5x on RLS-touching work.
2. **Supabase Auth custom_access_token_hook.** Documentation thin. Codegen often produces the older `raw_user_meta_data` pattern that does not work for RLS. You will fight this once, then it sticks. Add 2 days.
3. **Storage RLS is separate from table RLS.** Materials, SOAP attachments, exports: storage policies live in their own surface. Codegen will write table policies and forget storage. Add 1–2 days.
4. **pgcrypto + Vault wiring.** Codegen reflexively writes `pgp_sym_encrypt('val', 'plaintext')`. You must enforce `app_secret('key_name')` via Vault every single time. This is a code-review discipline issue, not a codegen issue.
5. **Edge runtime vs Node runtime.** Codegen mixes them. Some libraries (Razorpay SDK, certain crypto helpers) do not run on Edge. You hit this exactly once per surface and it costs an hour each time.
6. **Clinical scoring functions.** PHQ-9, GAD-7, PCL-5, OCI-R, MSI-BPD, SCOFF, AUDIT-C all have edge cases (reverse-coded items, "difficulty" question on PHQ-9 not scored, AUDIT-C cutoffs differ by gender). Codegen produces scoring that *looks correct* and silently miscalculates 1–2 tools. **You must hand-verify against published scoring sheets, not against codegen's tests.** Codegen tests will pass against codegen's understanding.

4 • Detailed estimation table

Assumptions: senior dev (Akash), spec already locked, Claude Code / Cursor with model parity to current Opus tier, Supabase Mumbai project provisioned, Vercel or Cloudflare Pages deploy target, Vault configured, ~25 hours/week of actual coding time. Working day = 5 productive hours net of context switches. Estimates exclude calendar-blocking external dependencies (covered separately in §5).

Work surface	Codegen leverage	Estimate (dev-days)	Why this number, what bites
Project scaffold — Next.js 14 App Router, TS strict, Tailwind, Supabase client, Sentry with PHI redaction, Cloudflare WAF + Turnstile, Vercel/Pages deploy, GitHub Actions CI, env management, <code>supabase gen types</code> pipeline, ESLint + Prettier + Husky	VERY HIGH	2–3	Most compressed work in any stack. Codegen ships a clean day-one repo. Risk is over-eager scaffolding (codegen adds boilerplate you do not need). Reject auth boilerplate that does not match Supabase patterns.
34-table schema + migrations — all 9 domains, GENERATED columns (<code>age_band</code> , SLA deadlines), CHECK constraints, FK ON DELETE RESTRICT , cross-domain FKs deferred to <code>0010b</code> , identity bridge (<code>users.auth_user_id</code> → <code>auth.users.id</code>), <code>current_user_id()</code> helper, idle-timeout helper	HIGH (DRAFTING), LOW (AUDIT PASS)	5–7	Codegen drafts each table from <code>data-model.md</code> §6 cleanly. You then read every line because codegen quietly drops a CHECK or weakens a NOT NULL when a downstream test fails. The audit pass is the work, not the typing.
RLS policies — per-table template, 6 worked examples, service_role-only audit INSERT (A4), SOAP body deny (B6), assessment trigger blocking patient assignment of clinician-only tools (A15), redaction rule for <code>session_notes</code> S/O/A/P and <code>conversations.ai_summary</code> in audit JSONB (A5)	HIGH (DRAFTING), MEDIUM (CORRECTNESS)	4–6	Drafting is fast. "Why is this query returning empty" debugging is the famous time-eater. Codegen's reflex is to broaden the policy; you must enforce narrow-first-prove-case. Single biggest source of unplanned hours.
RLS attack test suite — cross-patient, cross-MHP, role-escalation, admin-reads-SOAP, patient-reads-other-org, addendum-bypass, audit-actor-forgery	MEDIUM	2–3	Codegen writes the tests; you must invent the adversarial cases yourself. Codegen will not propose attacks it has not seen. This is where you spend half a day thinking like a malicious admin.

Work surface	Codegen leverage	Estimate (dev-days)	Why this number, what bites
pgcrypto + Vault + key custody — Vault setup, <code>app_secret(key_name)</code> helper, DM5 columns (<code>users.phone_encrypted</code> , MHP bank fields, safety-contact phone), service-role-gated decrypt views, encrypted- column read helpers	LOW	2–3	Vault setup is a UI-and-migration dance, doc-thin. Codegen helps minimally. Most of this is reading the Supabase Vault docs and getting the policy on <code>vault.decrypted_secrets</code> right.
Audit log — per-row sha256 trigger, monthly partitioning via <code>pg_partman</code> , JSONB <code>before_row / after_row</code> with redaction rule, weekly Merkle build via <code>pg_cron</code> , S3 ap- south-1 export with object-lock, V1.5 R2 mirror plan stub	MEDIUM-LOW	4–6	Codegen writes a working trigger and a passable Merkle builder. The redaction rule is where it will silently leak SOAP into JSONB if you do not catch it in review. <code>pg_partman</code> setup has its own learning curve. Tested-and-verified weekly Merkle externalisation is a half- day in itself.
Auth — Supabase Auth + <code>custom_access_token_hook</code> injecting <code>user_role</code> , identity bridge resolution, email-OTP for patients, TOTP enrolment for MHP/admin/counsellor, idle timeout enforcement, <code>HttpOnly</code> cookies, password-less patient sign-in	MEDIUM	4–5	Codegen frequently produces the deprecated <code>raw_user_meta_data</code> pattern. You fight it once, then it sticks. TOTP recovery codes + lockout policy is a genuine half-day of careful work.
Intake prototype — 7-screen flow, port HTML prototype to React, state machine, 8-cluster routing, severity-wins tiebreak, C-SSRS items 1–2 universal + 3– 6 gated, T5 short-circuit, JC alert dispatch, EN+HI i18n scaffold, <code>localStorage</code> draft with 7-day expiry, Cloudflare Turnstile, per-IP rate limit	VERY HIGH	7–10	Canonical React + state-machine work. Codegen excels. Bumped because of CF feedback loops on routing tables. If CF is responsive (1-day turnaround), lower bound holds; if 3-day turnaround, upper.
5-tester intake study — recruit, run sessions, transcribe, fix, re- run §10 cases against v2	NONE	5–7 calendar days, ~10h yours	Pure human work. Cannot compress. Do not gate other work on this — run in parallel during sprint 3 or 4.

Work surface	Codegen leverage	Estimate (dev-days)	Why this number, what bites
12 assessment tools — DB-driven rendering from <code>assessment_tool_versions</code> , version-locked instances, pure-function scoring, EN+HI content, A/B/C validation tier flag, milestone re-administration scheduler, A15 trigger blocking patient assignment of clinician tools	LOW (SCORING), HIGH (UI)	8–10	Each tool has scoring edge cases (PHQ-9 difficulty question, GAD-7 simple sum, AUDIT-C gender cutoffs, PCL-5 cluster scoring, MDQ multi-criteria positivity, ASRS Part A vs B, SCOFF threshold, OCI-R subscales, MSI-BPD threshold, ISI threshold, PSS-10 reverse coding, C-SSRS gating). Codegen produces plausible scoring that miscalculates 1–2 tools. Hand-verify against published sheets.
Hindi clinical content review by CF	NONE	1–2 calendar weeks, blocking	Pure CF time. Cannot compress with codegen. Plan to start CF review early so it overlaps build, not gates it.
MHP availability + booking — weekly recurring slots, block-outs, IST timezone handling, self-serve booking UI, session packages (4-session / 8-session), reschedule + cancel with policy enforcement	HIGH	5–7	Calendar/availability is well-trodden. Edge cases: DST (none in IST, win), MHP timezone if MHP travels (defer to V1.5), recurring-slot exception model (codegen handles cleanly).
Razorpay integration — order create, checkout, webhook receive + signature verify, idempotency keys, reconciliation cron, refund API, paise everywhere, display formatter at every UI surface, GST/TDS deductions ledger	MEDIUM	6–8	Codegen writes the happy path in 2 days. Webhook idempotency, retry-with-backoff, partial refund handling, dispute/chargeback hooks, signature timing-attack-safe compare — these need hand-design and a careful afternoon each. Live activation calendar-blocking (separate).
Earnings ledger — full attribution per transaction (service / PG charges / GST / TDS / commission / net / payout status), admin payout config UI, per-MHP rollups, refund flow ledger entries	MEDIUM	3–4	Schema is already specced. Codegen writes the rollup queries. Reconciliation edge cases (refund of a session in a paid-out batch) need you.
Daily.co + Jitsi — room minting on booking, signed-URL JIT mint server-side, Jitsi fallback toggle, session state machine (scheduled/in-progress/completed/no-show), pre-session 1-tap mood rating, post-session disconnect handling	MEDIUM-HIGH	4–5	Daily.co API is clean and codegen-friendly. Jitsi fallback adds a configurable provider abstraction (a half-day). Mood-rating UX is one screen.

Work surface	Codegen leverage	Estimate (dev-days)	Why this number, what bites
SOAP editor — signed + versioned + immutable-after-sign trigger, addendum pattern (<code>is_addendum + addendum_to_id</code>), B6 RLS denial enforcement, admin metadata-only view, MHP-only S/O/A/P read	MEDIUM	4–5	Codegen writes the editor in a day. The immutability trigger + addendum + B6 view layer is a focused half-week. The temptation to "soft-edit before sign" must be resisted at the schema level.
Medication list + ICD-11 + episodes — structured medication list (not e-prescription), ICD-11 lookup integration (WHO API or static catalog import), episode-of-care assignment, DM4 manual-close UX, 90-day-inactive admin widget	HIGH (UI), MEDIUM (ICD-11 INGEST)	4–5	ICD-11 catalog import is a one-time data engineering task; the WHO API has rate limits, prefer static import for V1. Episode lifecycle is straightforward.
Patient dashboard — upcoming sessions, history, materials library (Supabase Storage with signed URLs and storage RLS), safety plan editor, follow-up questionnaire scheduling (PHQ-9 / GAD-7 at weeks 0/2/4/8/12), assessment results display	HIGH	4–5	Storage RLS is the gotcha (separate policy surface from table RLS). Codegen forgets it twice before you internalise the discipline.
Informant linkage L1/L2/L3 — link request flow, consent capture, RLS for cross-account read with role flag, revocation flow, audit on every cross-account access	MEDIUM	3–4	RLS gets gnarly here — informant role is not a fixed <code>user_role</code> , it is a relationship. Either model as a separate policy clause per table, or via a helper function <code>can_read_patient(patient_id)</code> . Latter scales better.
MHP dashboard — today, week view, earnings, utilisation rate, client list with last/next session and care track and safety flags, assessment admin, track assignment (system suggests, MHP confirms)	HIGH	4–5	Standard dashboard. Codegen-friendly with shadcn/ui or tremor.so charts.

Work surface	Codegen leverage	Estimate (dev-days)	Why this number, what bites
Admin portal — MHP onboarding, booking oversight + calendar, revenue dashboard (aggregate + per-MHP), earnings ledger view, refund processing, incident/safety queue, deductions config, policy config (reschedule/cancel/refund), WhatsApp/email template editor	HIGH	8–12	Largest single surface. Honest call: consider <code>refine.dev</code> or <code>shadcn-admin templates</code>. Codegen pairs well with them and compresses this to 5–6 days. Without a template, 8–12 of hand-built React.
Notifications — MSG91 WhatsApp templates (booking confirm, 24h reminder, post-session check-in, safety net), template registration with Meta, one-way send, email via Resend/SES, retry with dead-letter queue, opt-out tracking	HIGH	3–4	Code is small; Meta template approval is calendar-blocking (1–3 weeks separate).
DPDP compliance plumbing — DSR portal (30-day SLA workflow, schema → operator queue → fulfil), consent ledger UI with versioned transcripts, breach 72h workflow + <code>incident_log</code> , vendor DPA registry CRUD with renewal alerts (A12 GENERATED <code>renewal_alert_at</code>), patient-queryable <code>phi_access_events</code> view (schema in V1, UI V1.5)	MEDIUM	5–7	Schema is already specced. The operator-side UI and SLA timer logic is real work. Easy to under-estimate because it is not glamorous.
Accessibility audit (WCAG AA) — axe-core in CI, manual screen-reader pass on intake (VoiceOver + NVDA), keyboard-only nav verification, contrast verification, label coverage	LOW	3–4	Codegen catches obvious misses (missing labels, missing alt). Manual screen-reader pass is the work. Plan a half-day with a screen reader you have not used before.
Pen-test / security review — \$7 walk + RLS attack tests run against staging, auth bypass attempts, Razorpay signature verification edge cases, Daily.co URL leakage check, secret-in-logs check, dependency CVE scan	NONE	5–7 calendar days external, ~₹40–80k	Either external reviewer or you with fresh eyes (worse but possible). External strongly recommended for a clinical/PHI product. Saday spec mandates W10 Tue security review anyway.

Work surface	Codegen leverage	Estimate (dev-days)	Why this number, what bites
Production cutover — Supabase Mumbai prod project, run full migration set, schedule cron jobs, configure Sentry PHI redaction, enable Cloudflare WAF + Turnstile in prod, verify backups + restorability, write runbook, set up status page	MEDIUM	3–4	Codegen helps with the migration script and runbook template. Backup-restore drill is the part you must actually do, not stub.
TOTAL — solo agentic, dev-days only (excluding calendar-blocking)	—	~110–140	~22–28 calendar weeks at 25h/week (5h/day net of context switches)

5 · Calendar-blocking dependencies (uncompressible)

These run in parallel with build, but the *launch date* is gated by the slowest one. Codegen does not move them.

Dependency	Calendar cost	How to de-risk
CF clinical sign-off on intake routing + 12 tool scoring + Hindi content	2–4 weeks aggregate (overlapping with build)	Start CF review on intake-spec v1.0 immediately (already drafted). Schedule weekly 1h CF review slot.
MSG91 + Meta WhatsApp template approval	1–3 weeks (Meta review opaque + iterative)	Submit templates in week 1, before any code is needed. Build with email + SMS fallback so launch does not gate on Meta.
Razorpay live activation + KYC	3–10 business days	Submit KYC docs in week 1. Build against test mode. Smoke test live with a ₹1 transaction at week 18.
Daily.co plan + recording config	1–2 days	Trivial. Sign up week 1.
Supabase project provisioning ap-south-1 + Pro plan + Vault enabled	1 day	Trivial. Day one.
5-tester intake study + revisions	5–10 calendar days	Run between sprint 2 (prototype done) and sprint 3 (port to backend). Recruit testers in week 1.
Pen-test / external security review	1 calendar week (firm) + 3–5 days of fix work	Schedule firm 8 weeks ahead of intended launch. Indian firms with healthcare experience: Lucideus, Network Intelligence, Payatu — quote ranges ₹40k–₹1.2L.
DPDP DPO appointment + Privacy Policy + Terms (telemed-aligned)	1–2 weeks legal review	Engage a DPDP-fluent counsel in week 1. Mental-health-specific clauses (MHA §23/§24) need a healthcare-fluent lawyer, not a generic startup lawyer.
Vendor DPAs signed (Supabase, Cloudflare, Razorpay, MSG91, Daily.co, Sentry, Resend/SES)	2–4 weeks aggregate	Most have standard DPAs accessible online. Razorpay's may need a back-and-forth. Capture in <code>vendor_dpas</code> table from day one.
Bank account + GST + TDS deduction setup for payouts	1–2 weeks	Founder admin work, parallel.

Aggregate calendar reality. Even at maximum codegen throughput on dev work, the project is gated by the slower of (a) ~22–28 build weeks at 25h/week, (b) ~6–8 calendar weeks of external/CF dependencies if started in parallel from day 1. Build is the longer pole at this team size, but if external work is started *late*, it becomes the gate. Start the calendar-blockers in week 1, not week 18.

6 • Aggregate cost view

Cost line	Solo agentic	Hybrid (you + 1 contractor)	Vendor as quoted	Vendor renegotiated
Your time (25h/week)	24–28 weeks	10–12 weeks	2–3 weeks (review only)	4–6 weeks (review only)
Cash to dev/vendor	₹0	₹2–3L	₹2L	₹3–5L
Infra (24 months runway)	~₹3.6–6L (₹15–25k/mo Supabase Pro + Vercel + Cloudflare + S3 + Sentry + Daily.co)	same	~₹0.7–1.4L (₹3–6k/mo VPS) — under-spec	same as solo if Postgres-mandated
Pen-test	₹40–80k	₹40–80k	not included	negotiate inclusion
Legal (DPDP DPO + privacy/terms)	₹50k–1.5L	₹50k–1.5L	₹50k–1.5L	₹50k–1.5L
CF time (clinical sign-off)	~30–50h	~30–50h	~30–50h	~30–50h
Maintenance (post-V1)	your time, ongoing	retainer ₹40–60k/mo or insourced	included 6 months, then per-feature	negotiate 12 months
Cash year-1 ballpark	~₹4.5–8L	~₹6.5–11L	~₹3–5L (under-spec, fragile)	~₹6–8L

What this table does not show: opportunity cost. Six months of your time at 25h/week is ~600 hours. At any reasonable clinic-revenue equivalence, that is materially more than the cash differential between solo and vendor. Pure-cash comparison flatters the solo path; total-cost comparison flatters the vendor path. Neither is wrong.

7 · Internal audit — where these estimates are wrong

7.1 · Where I am probably *under*-estimating

1. **RLS debugging tax.** Every senior Postgres-on-Supabase dev reports the same time-eater. Estimates assume 1.2x overhead on RLS-touching work; reality may be 1.5x. **Add ~3–5 dev-days.**
2. **Codegen review fatigue.** When you accept 80% of generated code, the 20% you do not catch compounds. After ~6 weeks of solo agentic dev you are rubber-stamping. Mitigation: schedule one "no-codegen review day" per week where you re-read what you accepted last week with fresh eyes. This costs ~10% time and saves the project. **Add ~10% across the board.**
3. **The two-hat problem.** 25h/week assumes you do not lose Wednesdays to clinic emergencies. You will. **Apply a 30% calendar buffer to anything quoted.** 24 calendar weeks → 31 calendar weeks realistic with clinic load.
4. **Custom_access_token_hook + RLS interaction with Realtime / Edge Functions** is documented thinly and codegen will mis-pattern it. **Add 2 days.**
5. **Supabase Storage signed-URL handling for materials + SOAP attachments** has subtle RLS interplay. Storage policies are separate from table policies. **Add 1–2 days.**
6. **MSG91 template approval cycles** are calendar-blocking and Meta is famously inconsistent. Do not gate launch on this — fall back to SMS for V1 launch. Calendar risk, not dev-day risk.
7. **Vault key-rotation playbook.** First time you rotate a Vault secret in production with encrypted columns referencing it, you will lose a day. Plan for it before V1, do not discover it after.
8. **The 5-tester study almost always surfaces a routing or wording change.** Budget 2 days of revisions, not 0.

7.2 · Where I am probably *over*-estimating

1. **Intake prototype.** You already have `intake-spec.md` at "engineer can scaffold from this" detail. Codegen could eat this in 4–5 days flat for V1, not 7–10. Bumped because of CF feedback loops; if CF is responsive, lower bound holds. **Subtract ~2 days.**
2. **Earnings ledger / payout config.** The schema in `data-model.md` is so specific that codegen has very little room to mis-interpret. Could be 2 days, not 3–4. **Subtract ~1 day.**
3. **Admin portal.** With `refine.dev` or `shadcn-admin` templates + codegen, this could compress to 5–6 days, not 8–12. **Subtract ~3 days if templating.**
4. **Production cutover.** If staging mirrors prod (which it should), cutover is a half-day, not 3–4. **Subtract ~1–2 days.**

7.3 · Net of corrections

Optimistic centre with corrections: ~110 → ~115 dev-days (over-estimation savings of ~7 days, under-estimation additions of ~12 days; net +5).

Realistic centre with corrections: ~140 → ~150 dev-days. Calendar with two-hat 30% buffer: **~28–31 calendar weeks. ~7 calendar months from kickoff to launch.**

This tracks `Saday_Action_Plan_by_Claude.md`'s 24-week Phase 0–4 plan, which Akash drafted *before* assuming agentic dev. The Action Plan's calendar was not bottlenecked on typing speed in the first place. It was bottlenecked on CF time, clinical sign-off, legal review, and 5-tester studies — none of which codegen accelerates. **Agentic dev compresses build, not project.**

8 • Risk register

Risk	Likelihood	Impact	Mitigation
Clinic emergencies eat coding weeks	High	High	30% calendar buffer baked in. Hard rule: no client deadline tied to a specific week. Buffer + monthly slip review.
Codegen review fatigue → rubber-stamping	High	High	Weekly "fresh-eyes review day." Pair-review with Claude Code on diffs older than 3 days. Reject "make the test pass" patches.
RLS broaden-on-fail anti-pattern	High	High	Discipline rule: every RLS change needs a passing attack test in the same PR. CI gate: <code>RLS_ATTACK_TESTS</code> required.
Clinical scoring miscalculation in 1–2 tools	Medium	Critical	Hand-verify every tool against a published scoring sheet. CF re-verifies before V1 launch. Lock <code>assessment_tool_versions</code> with a version per (tool, version, language).
SOAP body leak via storage RLS oversight	Medium	Critical	Storage policies tested separately. Adversarial test: admin user attempts to fetch a SOAP attachment URL.
Pen-test surfaces a critical finding 2 weeks before launch	Medium	High	Schedule pen-test at week 18, not week 26. Buffer 2 weeks for fixes.
Meta WhatsApp template rejection cycles delay launch	Medium	Medium	Submit week 1. SMS fallback for V1. Do not couple launch to WhatsApp.
Vault key custody bug → unrecoverable encrypted columns	Low	Critical	Test key rotation before V1 with synthetic data. Document recovery runbook. Backup Vault export to a separate secure store.
Bus factor = 1 (you alone leaves / falls ill)	Low	Critical	Documented runbooks. Spec is exhaustive (already true). Hybrid path materially de-risks this.
Razorpay webhook idempotency bug → double-charge / double-refund	Medium	High	Idempotency keys + persisted webhook ledger. Reconciliation cron daily. Manual refund-of-double review pre-V1.
DPDP DSR request arrives in V1 and the operator UI is V1.5	Low	Medium	Schema is V1; operator-side fulfilment can be manual via SQL for first 90 days post-launch. Document.
Hindi content not CF-signed by launch	Medium	Medium	Launch EN-only if needed; HI as fast-follow. Do not block launch on translation.
Cloudflare Turnstile + per-IP rate limit too aggressive → real users blocked from intake	Low	Medium	Generous limits in V1; tune down only after observing traffic. Log challenge solves to detect false positives.

Risk	Likelihood	Impact	Mitigation
Audit Merkle externalisation cron fails silently for weeks	Medium	High	Cron failure → Sentry alert + dashboard widget showing last successful Merkle anchor timestamp. Alert if >8 days.

9 · Solo build path — week-by-week skeleton

28-week plan with 30% buffer. Two-hat weeks (25h coding) assumed. Calendar-blocking work runs in parallel from week 1.

Week	Build focus	Parallel calendar work	Output
1	Project scaffold, Supabase Mumbai, Vault, types pipeline, Sentry, Cloudflare, CI/CD	Submit MSG91 templates · Razorpay KYC · engage DPDP counsel · book pen-test for week 22 · CF review of intake-spec begins	Repo + deploy pipeline + dev/staging envs
2	Migrations days 1–4: domains 1–4 (users, mhps, patients, episodes)	CF intake review continues	Half schema in DB
3	Migrations days 5–8: domains 5–9 + cross-domain FKs (0010b) + Vault wiring	Vendor DPAs signed: Supabase, Cloudflare, Daily.co	Full schema + Vault
4	RLS policies for users, patients, mhps, episodes + first attack tests	Vendor DPAs: Razorpay, MSG91, Sentry	Half RLS surface + 6 attack tests passing
5	RLS policies for session_notes (B6), assessment_*, audit_log + remaining attack tests	CF clinical sign-off on cluster taxonomy + routing	Full RLS + 12 attack tests
6	Auth: Supabase Auth + custom_access_token_hook + identity bridge + email-OTP for patients + TOTP for MHP/admin	Privacy policy + ToS draft v1	Login flows for all 3 roles
7	Audit log triggers + monthly partition + redaction rule + Merkle build cron + S3 export	5-tester recruitment	Audit log live + first Merkle anchor in S3
8	Intake prototype port (S0–S3) + EN+HI scaffold + localStorage draft	CF Hindi review starts	Half intake live
9	Intake S4–S6 + 8-cluster routing + C-SSRS gating + T5 short-circuit + JC alert	5-tester study runs	Full intake live
10	Intake revisions from 5-tester study + Turnstile + rate limits + server-side routing port	Razorpay live activation arrives	Intake v2 with study fixes
11	Assessment engine: PHQ-9, GAD-7, MDQ, ASRS-v1.1 (rendering + scoring + version locks)	CF verifies PHQ-9, GAD-7 scoring	4 tools live
12	Assessment engine: AUDIT-C, PCL-5, OCI-R, MSI-BPD	CF verifies these 4	8 tools live
13	Assessment engine: SCOFF, PSS-10, ISI, C-SSRS + milestone scheduler	CF verifies last 4 + Hindi sign-off batch 1	All 12 tools + milestones

Week	Build focus	Parallel calendar work	Output
14	MHP availability + booking + reschedule/cancel	—	Booking flow live
15	Razorpay: order + checkout + webhook + idempotency + signature	—	Test-mode payments end-to-end
16	Razorpay: refund + reconciliation + earnings ledger	—	Earnings live
17	Daily.co + Jitsi + session state machine + pre-session mood	—	Video sessions live
18	SOAP editor + signing trigger + addendum + B6 verification	Pen-test firm intake call	SOAP live, B6 attack tests pass
19	Medication list + ICD-11 + episode-of-care UI + DM4 manual close	—	Clinical surfaces complete
20	Patient dashboard + materials library + safety plan + follow-up scheduling	—	Patient surface complete
21	Informant linkage L1/L2/L3 + cross-account RLS	Pen-test starts	Informant flows live
22	MHP dashboard + admin portal scaffold (refine/shadcn-admin)	Pen-test report received	MHP dashboard + admin shell
23	Admin portal: revenue + earnings + refund + incident queue	Fix pen-test findings	Admin core surfaces live
24	Admin portal: configs (deductions, policies, templates) + DSR portal + consent ledger UI	Pen-test re-verify	Compliance plumbing live
25	MSG91 + email + retry + dead-letter + opt-out	WhatsApp templates approved (best case)	Notifications live
26	Accessibility audit + axe-core CI + manual screen-reader pass + remediation	—	WCAG AA verified
27	Production cutover + backup verify + runbook + monitoring + load test	Bank + GST + payouts ready	Production env hot
28	Soft launch with 5–10 invited patients · monitor · fix · go-public on day 7	—	Launch

10 • Decision rubric

Pick a path against these 7 questions. Each answer biases toward one column.

Question	Solo agentic	Hybrid (recommended)	Vendor (renegotiated)
Will you have ≥25h/week to code for 28 weeks straight?	Required	10–12h/week is enough	2–3h/week (review only)

Question	Solo agentic	Hybrid (recommended)	Vendor (renegotiated)
How much do you want to learn this codebase intimately?	Maximally	Deeply on safety-critical, lightly elsewhere	Spec-level only
Cash budget for build (year 1)?	Lowest	Mid	Higher (if renegotiated)
Tolerance for bus-factor=1?	High	Medium	Low
Confidence the spec is right and a vendor will not get it?	Highest preserved	You guard safety-critical surfaces	You spec-review every PR
How much do you trust agentic codegen on PHI code?	With heavy review	Same, less surface	You delegate to vendor's review
Do you want to ship faster than 6 months?	No (24–28 weeks)	Yes (14–16 weeks)	Yes (12–20 weeks)

Cumulative scoring

Solo path scores best when

- Cash is the binding constraint
- You want maximum codebase ownership
- Bus-factor concerns can be deferred
- Clinic load is genuinely 50%-of-week or less for 7 months
- You expect to keep building this product yourself for 1–2 years

Hybrid path scores best when

- Speed-to-launch matters (14–16 weeks)
- You can identify a competent contractor (₹1.5–2.5k/hour senior, Indian Next.js+TS pool, Postgres-fluent)
- Safety-critical surfaces are non-negotiable in your hands
- Cash budget is mid (₹6–11L year 1 acceptable)
- You want a clear handoff boundary (you do schema/RLS/audit/intake/scoring; contractor does dashboards/payments/video/notifications)

My honest read. Solo agentic is a credible and respectable path on this stack — more credible than on any other stack you could have picked. But the project is gated by CF time and external dependencies, not coding speed, so the gain over hybrid is smaller than it looks. The hybrid path lands ~10–12 weeks earlier with the same correctness guarantees on the load-bearing surfaces, and a ₹2–3L contractor spend is <1.5× a single month of clinic revenue equivalence. **Hybrid is the rational default. Solo is the right call if cash is the binding constraint or if owning every line is a hard requirement for you.**

11 • Open questions for Akash to answer before locking a path

1. **Honest 8-week-ahead clinic load forecast.** Will you genuinely have 25h/week for 7 months, or is it 15h/week with growth into 25? The latter pushes solo to ~10 calendar months.
2. **Contractor pool reality.** Do you have a known Next.js+TS+Postgres senior in your network, or do you need to recruit cold? Recruiting cold adds 4–6 weeks before contractor work starts and is a hidden risk.
3. **What is the launch trigger?** First 10 patients? First MHP onboarded? First DPDP-compliant audit-ready checkpoint? The trigger changes which surfaces are V1 versus deferred.
4. **Hindi at launch — required, or fast-follow?** If fast-follow, you save 1.5–2 calendar weeks of CF blocking time on launch.

5. **Are you willing to ship V1 EN-only with HI 4 weeks later?** Decoupling translation from build calendar is a strong simplification.
6. **Is admin portal a V1 must-have or V1.1 acceptable?** If admin can be SQL + a Notion runbook for 60 days post-launch, you save 2 weeks of dev. Patients and MHPs cannot wait; admin can.
7. **Solo dev review cadence.** Are you willing to enforce a weekly fresh-eyes review day at the cost of ~10% throughput? If not, expect the failure mode to bite around week 14.

Drafted: 2026-05-05 by Claude (Opus 4.7) for Akash, in response to a request to compare the vendor quote against a solo + agentic path on the locked stack.

Status: internal estimating document. Numbers are honest best-guesses with explicit over/under bias flags in §7.

Companion documents: `data-model.md` v1.1 · `intake-spec.md` v1.0 · `stack-decision.md` v1.0 · `Saday_Action_Plan_by_Claude.md` · `Saday_Master_Blueprint.html` · `Saday Wellness Quote.pdf` (vendor)

Reading order if first time: §1 → §6 → §7 → §10 → §9 → §2/§3 → §11.